Journal of Computer Virology and Hacking Techniques (2026) 22:8
https://doi.org/10.1007/s11416-025-00578-w

RESEARCH

From comments to threats: leveraging NLP to detect malicious browser extensions

Adam Flemark[1] · Axel Paulander[1] · Pablo Picazo-Sanchez[1]

Received: 22 February 2025 / Accepted: 15 September 2025
© The Author(s) 2025

Abstract
With the increasing popularity of browser extensions, there is a growing concern about malicious actors exploiting this software to distribute malware. Existing solutions rely on manual review processes or traditional methods like static, dynamic or hybrid analysis, which are insufficient in detecting complex and evolving threats. In this study, we investigate the potential of Natural Language Processing (NLP) for automatically classifying users' comments in the Chrome Web Store, the public repository where extensions are stored and distributed, to identify malicious extensions. We propose a novel framework called CoTH that leverages NLP techniques to analyse the textual feedback provided by users and detect patterns indicative of malicious activity. We evaluate the accuracy of our model using a dataset of user reviews and demonstrate its effectiveness in identifying malicious extensions. Our findings suggest that NLP-based comment analysis can be a valuable addition to existing security measures, providing an opportunity for more accurate and efficient detection of malware in the Chrome Web Store.

Keywords NLP · Web security · Browser extensions · Malware

1 Introduction

Internet browsers are popular software applications included in most operating systems. Browsers like Google Chrome, Mozilla Firefox, and Microsoft Edge allow users access to near-limitless information and entertainment. The sheer versatility required to cater to every user is a futile exercise for browser developers, leading to the creation of third-party browser extensions. Extensions enable users to customise their browsers to suit their needs and preferences by adding additional features, such as Artificial Intelligence (AI) writing assistance, scraping the web for possible discount coupons or website translation tools.

Most popular browser vendors curate available extensions through their repositories to enable users to discover and install new extensions easily. Google Chrome, the world's most used browser [47], stores all extensions in a repository named the Chrome Web Store (CWS). However, the CWS is not simply a repository of extensions but a fully-fledged online storefront complete with categories, charts of the most popular extensions, and personalised recommendations. To present users with relevant suggestions, the CWS implements a Collaborative Filtering Recommendation System (CFRS) by letting users rate and review extensions on their platform to share their experiences and opinions with other users [19].

Installing extensions from the CWS is a straightforward procedure. Users must press the install/add button on the desired extension download page. Also, if applicable, users must approve any permissions the extension requires in a separate pop-up window. Therefore, personalising one's browser to fit one's preference is reasonably straightforward. This "app within an app" perspective may make users complacent and overlook the security implications of installing extensions that have partial access to their computer and could be malicious or otherwise make them vulnerable to malware on the web [7, 46, 51, 62].

Malware among browser extensions comes in many forms, as an extension can not only access the users' devices

✉ Pablo Picazo-Sanchez
ppicazo@hh.se

Adam Flemark
adam.flemark@gmail.com

Axel Paulander
axel.paulander@gmail.com

[1] Halmstad University, Halmstad, Sweden

Published online: 19 December 2025

but also inject arbitrary HTML and JavaScript into the web pages that the users visit. These capabilities allow malicious extensions to log keystrokes to steal login information [62], inject malicious advertisements [46], harvest computation power for botnets [7] and more. According to Kaspersky, these malicious browser extensions affect millions of users, having around 87 million downloads from just the CWS [51].

Researchers are addressing the challenge posed by malicious actors [14, 17, 30, 39]. However, there is no one perfect solution to prevent or immediately flag these malicious extensions from entering the CWS ecosystem and harming users. Although most researchers have proposed solutions based on static [28, 36, 44], dynamic [6, 34, 50], or hybrid analysis [45, 56, 58], recent researchers have proposed solutions by analysing the metadata that Google publicly shows in the CWS, like the pattern of the number of downloads [39], or the time overlap of the users' comments [33].

An alternate avenue of metadata analysis is Natural Language Processing (NLP), which enables researchers to use the large corpora of text available in the metadata to detect non-trivial patterns in text, making it possible to perform analysis using previously neglected data. With the advent of NLP, there is an overlooked question in malware detection of browser extensions, which motivates us to find the answer to the following research question:

> **RQ**: Is it possible to detect malicious extensions by analysing users' comments?

Inspired by recent research where authors demonstrated that users can easily manipulate the review system that the CWS implements by flooding the page with positive reviews and ratings to make their extension appear better and more prevalent than in reality [33], we propose CoTH (Comments To Threats), a framework that leverages NLP to classify browser extensions and to detect whether they are malicious by analysing the comments written by users in the CWS. To answer our research question, we evaluate CoTH using a dataset of over 1.7 million reviews from 1.5 million users on 55,000 extensions collected by scraping the CWS as of Feb 2024.

The rest of the paper is organised as follows. Section 2 presents some basic concepts and definitions. Section 3 details the source and structure of the dataset used. Section 4 covers our novel framework. We present the related work in Sect. 5 and conclude the document in Sect. 6.

2 Background

This section covers some of the concepts and general definitions necessary to fully understand this document. In particular, we focus on data augmentation and ensemble methods, two methods that we use throughout the document.

2.1 Data augmentation

Ideally, the datasets used for training Machine Learning (ML) models should be balanced, i.e., datasets that have the same number of instances for each class label. Such a balanced dataset is a common assumption in model development, but such a claim becomes challenging in real-world scenarios, especially in malware analysis. In fact, most datasets are imbalanced, with benign classes dominating the dataset, and it is most often the minority class (malicious samples) that is of interest [61].

When training a model on such imbalanced data, performance usually suffer due to biased learning [24]. As a consequence, traditional accuracy metrics like accuracy and precision may provide unrealistic values when applied to imbalanced datasets. To illustrate this point, consider a dataset comprising 90% instances of one class and 10% instances of another. If a model were to assume that all data belonged to the majority class, evaluating its accuracy on this dataset would yield an apparent score of 90%, which may seem impressive.

There are many ways to combat the issues brought on by imbalanced data, being *undersampling* and *oversampling* the most common strategies. When augmenting the dataset, it is most common to either remove instances of the majority class, known as undersampling, or conversely add instances of the minority class, known as oversampling. Crucially, oversampling does not include measuring new data, instead, new instances of the minority class are synthetically generated either by repeating known values or by generating new points using techniques such as Synthetic Minority Over-sampling (SMOTE) [11].

2.1.1 Undersampling

The goal of undersampling is to reduce the number of entries of the majority class to a number closer to that of the minority class. There are two major approaches to doing so: generation and selection.

In generation, undersampling is achieved by creating an entirely new dataset that is both balanced and aims to retain the structure of our data. This can be done by e.g., using k-means clustering on the original data and then creating synthetic data by randomly selecting several points inside clusters associated with each label.

Selection, on the other hand, reduces the dataset by selecting relevant samples using some heuristic. One example of selection aims to de-noise the decision boundary by removing pairs of points that are both of opposite classes and each other's closest neighbours, so-called Tomek links [52], or using a k-Nearest Neighbors (kNN) classifier trained on the whole dataset and then removing any point for which the classifier predicts the incorrect class [59].

2.1.2 Oversampling: translation and back-translation

The opposite approach to undersampling is oversampling, where the aim is to generate new data that is still highly relevant and realistic. For example, in image-related tasks, one might rotate some images. While it might look the same to human eyes, since the pixels are now in a new order, it is an entirely unseen image for a computer model. In text-related tasks, the aim is to change the phrasing but not the content of our data.

There are two popular approaches to do so: *back translation* and *paraphrasing*. In back translation, the idea is to translate the text into other languages and then back to the target language, hopefully resulting in a rephrased version of the original sentence. On the other hand, paraphrasing aims to rephrase the text more directly. While humans could do this, e.g., volunteers, this is often unrealistic due to many factors, including but not limited to: cost of labour, human inconsistency, and time constraints.

Therefore, back translation is widely used, specifically for training Large Language Models (LLMs) [9]. When applying data augmentations to specific data points, one can either do so ahead of time and append the new data to the dataset or apply augmentations on the fly during training using random probability.

2.2 Ensemble methods

Ensemble methods are a common approach in ML, combining multiple models to perform a specific task, such as classification [29]. By leveraging these models' diverse strengths and weaknesses, ensemble methods aim to improve overall performance through increased accuracy and reduced overfitting, where a model performs poorly on new, unseen data despite excellent performance on training data. When implementing ensemble methods, it is necessary to decide how to combine the multiple outputs produced by the models to achieve optimal accuracy in the final output. Some of the common combination strategies for classification tasks include majority voting, weighted voting, or averaging predicted probabilities. These approaches are sometimes called voting systems.

The most straightforward voting system is the majority vote, where the most common output from each model is used as the final ensemble output. However, this approach overlooks a crucial factor: certainty. In reality, models often produce floating point numbers between 0 and 1, which can be interpreted as the *model's certainty*. This certainty is a key consideration in voting systems, as it can influence the final ensemble output and the overall performance of the ensemble.

Two ways of incorporating certainty into the vote are the highest unstandardised and highest standardised value [38]. The highest unstandardised or standardised value works similarly, where the model's certainties for each label are summed, and the highest total certainty is picked for the final output. The difference between the two approaches comes from the preprocessing done on the certainties before taking their sum. While the highest unstandardised value uses the certainties as-is, the highest standardised approach first standardises the certainties using min-max scaling. Standardising the values results in what can be practically interpreted as a ranking from least to most certain.

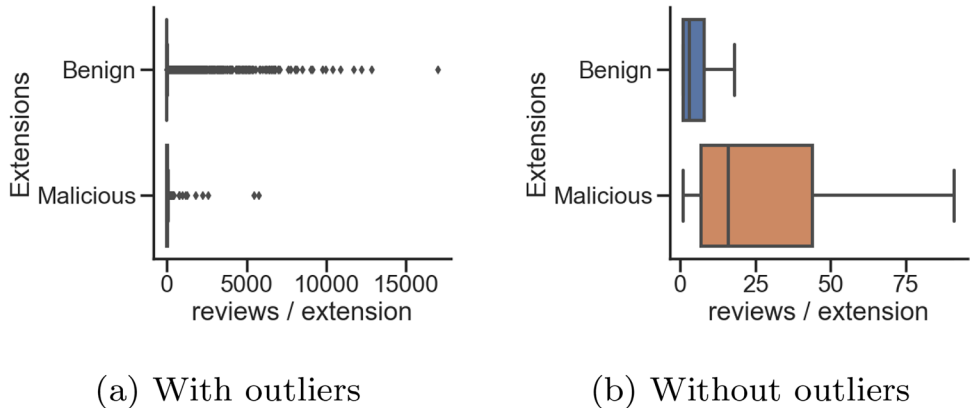
Table 1 Extensions' comments and metadata we downloaded from the CWS as of February 2024

Feature	dtype	Desciption
ExtensionID	String	Extension identifier
UserID	String	User identifier
Review	String	User review
Timestamp	int64	ms since Unix epoch
Rating	int64	User rating (1–5 stars)
Malware	Boolean	Class label (malicious/benign)

3 Reviews in the web store

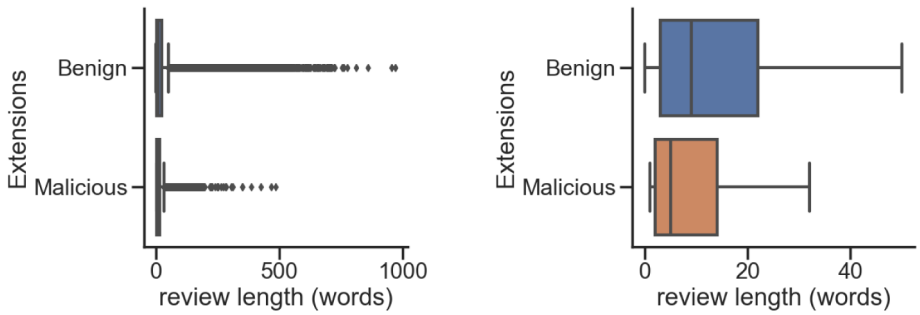
As of February 2024, we scrapped CWS and found over 120,000 browser extensions. Among them, not all extensions had reviews. In particular, we found that less than a half (over 55,000) had at least one review. In total, we downloaded 1.7 million reviews. For each review we got the `userID`; the `review` itself; the `rating` on a scale of 1–5 stars the user gave, and; when the review was written (`timestamp`). Additionally, for every one of the 55k extensions, we continuously queried Google for its maliciousness for 6 months. This `malware` label is the class label of the associated extension certified by Google following the same methodology as other researchers recently presented [31]. We include a summary of the information we downloaded in Table 1. To replicate the dataset used in our experiments, researchers may use alternative sources, such as https://chrome-stats.com, corresponding to the same time frame.

Imbalanced Dataset. In our dataset, 0.49% of the extensions are malicious (around 270 malicious extensions). Regarding reviews, over 1.91% (over 33,000 comments) of



(a) With outliers (b) Without outliers

Fig. 1 Distribution of number of reviews per extension as of February 2024 in the CWS



(a) With outliers (b) Without outliers

Fig. 2 Distribution of the length of reviews per extension in the CWS as of February 2024

the reviews are associated with malicious extensions, i.e., reviews posted by users on extensions that Google considers malicious. Furthermore, the number of reviews per extension is highly imbalanced, with most extensions having only a small number of reviews, as seen in Fig. 1.

More concretely, we find that around 79.31% of the extensions have fewer than ten reviews, while only around 20.69% of the extensions have more than ten reviews. We also find that the largest 13.6% of extensions by number of reviews contain about 90% of all reviews. This data imbalance makes training robust models across different extension types and review volumes challenging.

3.1 Malicious extensions and comments

A malicious extension is a blanket term for a browser extension designed to cause some negative effect on the user. These effects can range from illegal data collection and injection of unsolicited ads to modifying the web contents [55, 60].

On the other hand, we define a malicious review or comment as *a comment left intending to deceive others about the commented extension*. Depending on who is leaving the malicious comment, it could be false accusations of malicious behaviour to reduce popularity in the commented extension and increase popularity in another competing extension. Similarly, a malicious review might also be a false positive testimony on a malicious extension with the intent to increase perceived trustworthiness and increase the extension's popularity to reach more devices. Leaving malicious reviews of either type is known as "search rank fraud" [41].

While identification of malicious reviews by the above definition would no doubt prove valuable for malware detection as a whole, labelling such a dataset would be challenging and is, therefore, outside the scope of this research. Therefore, the maliciousness of a review is instead bound to the extension it is left on. All comments on benign extensions are considered benign; likewise, all comments on malicious reviews are considered malicious.

3.2 Clustering

To cluster comments, the first and most straightforward approach is to use both the number of reviews and the length of the reviews for the benign and malicious extensions as we can see in Figs. 1 and 2. With this simple-but-effective approach, we empirically demonstrate that both the number of reviews and their length exhibit some patterns that can be used as an indication for malware detection. The drawback is that it will be easy to bypass by malicious actors and circumvent any solution based on these metrics. In

other words, malware extensions might alter the number of reviews as well as the length so any detection algorithm solely based on these two metrics, will become useless.

Our second approach is to get the Part-of-Speech (POS) tags of all reviews, converting each part in a review to the name of its function or word class and later, we apply Hierarchical Density-Based Spatial Clustering of Applications with Noise (HDBSCAN), a clustering method, to group reviews from malicious and benign extensions based solely on their content.

Table 2 POS analysis using `en_core_web_lg` model from Spacy of a malicious cluster composed of 10 extensions

Extension ID	POS Elements
gfbgiekofllpkpaoadjhbbfnljbcimoh	['DET', 'ADJ', 'NOUN', 'PRON', 'VERB', 'PRON', 'ADP', 'PRON', 'NOUN', 'PUNCT']
diapmighkmmnpmdkfnmlbpkjkealjojg	['DET', 'ADJ', 'NOUN', 'PRON', 'VERB', 'PRON', 'ADP', 'PRON', 'NOUN', 'PUNCT']
hdgdghnfcappcodemanhafioghjhlbpb	['DET', 'ADJ', 'NOUN', 'PRON', 'VERB', 'DET', 'NOUN', 'CCONJ', 'NOUN', 'ADP', 'DET', 'NOUN', 'PUNCT']
pbdpfhmbdldfoioggnphkiocpidecmbp	['DET', 'ADJ', 'NOUN', 'PRON', 'VERB', 'DET', 'NOUN', 'CCONJ', 'NOUN', 'ADP', 'DET', 'NOUN', 'PUNCT']
iicpikopjmmincpjkckdngpkmlcchold	['DET', 'ADJ', 'NOUN', 'PRON', 'VERB']
mgkbeabpcphiihfgjofinnlfcjjaagpe	['ADJ', 'NOUN', 'PRON', 'VERB', 'DET']
oakbcaafbicdddpdlhbchhpblmhefngh	['PRON', 'ADV', 'VERB', 'PRON', 'ADP']
dppnhoaonckcimpejpjodcdoenfjleme	['PRON', 'ADV', 'VERB', 'PRON', 'ADP', 'PRON', 'NOUN', 'PUNCT']
bkpdalonclochcahhipekbnedhklcdnp	['PRON', 'ADV', 'VERB', 'PRON', 'ADP', 'PRON', 'NOUN', 'PUNCT']
phepcklpbfinbfflpjnmehcnnoofbghp	['PRON', 'VERB', 'PRON', 'ADP', 'PRON', 'NOUN', 'CCONJ', 'PRON', 'AUX', 'PART', 'VERB', 'PUNCT']

For example, a "10/10 Would recommend!" review becomes `[number, verb, verb, punctuation]`. We then use Term Frequency–Inverse Document Frequency (TF-IDF) to convert our POS tags into numerical features. TF-IDF achieves this text-to-number conversion using Equations (1) to (3) where the variables are: a term t, a document d the frequency of t in d $f_{t,d}$ and the set of all documents D. In our case, a term is an n-gram of POS tags, and a document is a single review. This allows us to convert each review into a high-dimensional vector of numerical features where the position represents some n-gram present in the corpus, and the value is the TF-IDF score of that n-gram if it is present in the document or zero otherwise. After extracting our numerical features, we applied HDBSCAN to cluster our reviews.

$$\mathrm{tf}(t,d) = \frac{f_{t,d}}{\sum_{t' \in d} f_{t',d}} \tag{1}$$

$$\mathrm{idf}(t,D) = \log \frac{|D|}{|\{d : d \in D \text{ and } t \in d\}|} \tag{2}$$

$$\mathrm{tfidf}(t,d,D) = \mathrm{tf}(t,d) \cdot \mathrm{idf}(t,D) \tag{3}$$

For this experiment, first we filtered extensions with more than 5 reviews and less than 50, so we can remove all popular extensions that just add noise to the analysis. After that, we use Spacy, a well-known library in Python and more concretely, the `en_core_web_lg` pipeline,[1] to get the POS tags of all reviews. After the filtering process, we analysed over 4,300 reviews written in 306 extensions (136 benign and 136 malicious ones). We randomly used a subset since clustering the entire dataset does not yield any valuable information because of clutter and the entropy of user reviews, leading to large amounts of outliers. Finally, to cluster POS tags, we used HDBSCAN with a minimum size of cluster of 4 reviews. Out of the 306 extensions, HDBSCAN could not classify 101, i.e., over 33% of the extensions.

Table 3 gives a good sense of the homogeneity within the different clusters. By looking at the outermost cases, which tell us the number of clusters that only contain malicious or benign reviews, we can see that the combination of TF-IDF and HDBSCAN can pick up underlying patterns in the reviews left on browser extensions quite effectively.

After some manual analysis of the clusters, we could find a cluster composed of 10 malicious extensions (see Table 2). sharing the same reviews' structure among them all. Although we can see some patterns, the problem is that the clustering is far from perfect, as we can see in Table 3,

[1] https://spacy.io/models/en

where the perfect solution would be pure clusters, i.e., clusters entirely composed of either benign or malicious extensions. These numbers are even more spread when we use the entire dataset.

4 CoTH: from comments to threats

To answer our research question: "Is it possible to detect malicious extensions by analysing users' comments?", in this section, we present CoTH, a novel framework that leverages NLP to flag potential malicious browser extensions based on the comments that users write in the CWS.

4.1 Architecture

To enable the classification of extensions based on their reviews, CoTH implements a combination of LLM, traditional neural networks and voting systems. CoTH's architecture, see Fig. 3, implements four main components: data pre-processing, reliability scoring, classification and voting.

The initial reliability scoring system makes use of pre-trained LLM to generate additional features for later steps. These numerical features are referred to as "reliability" and "time-scaled reliability", covered in depth in Sect. 4.3.

The classification components use the raw comment as input to another pre-trained LLM. Then, the LLM's CLS[2] token and the previously generated numerical features are all passed through a fully connected layer to generate the final classification. Here, the classification is on a review level, as discussed in Sect. 3.1. CoTH does not require any specific LLM for the classification component as it is intended to go through some epochs of fine-tuning before making any inference. However, the selected LLM could have implications for the overall performance, covered in Sect. 4.6.

The last component, voting, is how CoTH can classify a single extension based on many reviews. Each review is passed through the classification component, and the outputs are then used as votes to classify the extension as a whole. Multiple voting systems can be used, like with the LLM of the classification component.

Table 3 Small subset of a cluster distribution produced by HDBSCAN. 0% means clusters solely composed by benign extensions whereas 100% are clusters entirely composed of malicious extensions

% of Malicious Reviews	0%	0–50%	50–80%	80–100%	Total
# of Clusters	36	87	30	61	192
% of Maliciousness	0%	44.6%	70.1%	99.7%	
# Extensions	79	234	117	156	586

4.2 Data pre-processing

For CoTH to work as intended, the data might require pre-processing. What kind of pre-processing is required depends on the LLM models chosen for the other components. If, for example, any model in the pipeline is only trained in a single language, all data must be translated into that language. For instance, "twitter-roberta-base-sentiment-latest", one of the top 4 most downloaded text classification models on Hugging Face with over 2 M downloads as of May 2024,[3] was trained on English texts, so if CoTH would use such a model, the reviews should be in English. Different strategies exist to overcome such constraints if an all-English dataset is unavailable. For example, translating all the reviews to English using external software or filtering out and keeping only the English comments.

Similarly, to prevent over- and underfitting issues brought on by imbalanced data, such data must be resampled before being used for training CoTH. For example, our dataset presented in Sect. 3 has about 1.91% malicious reviews—reviews written on malicious extensions. This is heavily imbalanced and would therefore require some resampling for CoTH, or any model for that matter, to fit correctly without over- or underfitting dramatically.

4.3 Reliability scoring

After the initial data pre-processing, the subsequent task of calculating reliability scores can start. CoTH does not require any specific models for this task. It uses the models

[2] stands for classify token.

[3] https://huggingface.co/models?pipeline_tag=text-classification&sort=downloads.

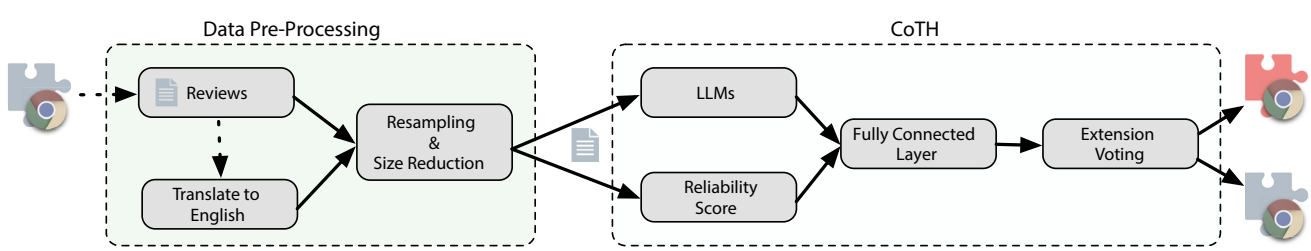


Fig. 3 CoTH model architecture and the data pre-processing stage

in a collective decision-making process whereby each model determines a reliability score for individual users by voting. We implement three different voting strategies: a majority vote, the highest unstandardised value, and the highest standardised value.

The most straightforward voting system is the majority vote, where the most common output from each model is used as the final ensemble output. However, this approach overlooks a crucial factor: certainty. Models often produce floating point numbers between 0 and 1, which can be interpreted as the *model's certainty*. This certainty is a critical consideration in voting systems, as it can influence the final ensemble output and the overall performance of the ensemble.

Two ways of incorporating certainty into the vote are the highest unstandardised and highest standardised values [38]. These values work similarly, where the model's certainties for each label are summed, and the highest total certainty is picked for the final output. The difference between the two approaches comes from the preprocessing done on the certainties before taking their sum. While the highest unstandardised value uses the certainties as-is, the highest standardised approach first standardises the certainties using min-max scaling. Standardising the values results in what can be practically interpreted as a ranking from least to most certain.

By classifying all known reviews, CoTH can then calculate the frequency of "genuine" reviews for each user according to our system; we refer to this value as the users "reliability". For the training set, all reviews in such a set contribute to calculating the reliability scores, but the same is not valid for the evaluation set. For the evaluation set, the reliability scores are instead calculated on a per-extension basis where, for each extension, only that extension and the training set are considered. This emulates receiving each extension in the evaluation set as new data to update the reliabilities of known and new users.

After getting the reliability scores for all users, CoTH appends the scores for a given user to all reviews left by that user. Then, it scales each score by the age of its associated review compared to the oldest and newest reviews on the same extension. This "time-scaled reliability" gives older reviews for an extension less influence than more recent reviews, which is essential as a user or extension could have been malicious in the past but no longer or vice-versa.

4.4 Classification

CoTH is agnostic of the LLM used in its construction. However, different LLM might yield different performance and characteristics. One LLM might result in a high precision with a low recall, and another LLM might give the opposite result. This allows CoTH to be modified to fit different malware detection pipelines with different model-performance preferences.

In the classification model, the chosen LLM receives raw user comments as input. Then, for each comment, the CLS token from the LLM is concatenated with a vector of the reliability and time-scaled reliability scores associated with that comment. This new feature vector is then fed to a fully connected layer, which attempts to classify the comments as malicious or benign according to the definition discussed in Sect. 3.1.

Although we call this component "classification", it is not the final step as it classifies single reviews as opposed to full extensions.

4.5 Voting

The voting component classifies what we call a full extension by aggregating its individual reviews. Each review classification from the previous component is regrouped according to its associated extension. Then, similarly to how CoTH generates the reliability scores, a vote takes place. The voting component uses the same voting schemes as reliability scores: majority vote, highest unstandardised and highest standardised.

Majority voting handles each review equally, i.e., labelling an extension as malicious if more than 50% of its reviews predict malware. Unstandardised probabilistic voting sums the raw model probabilities for each class, giving more influence to confident predictions. This leverages the model's inherent confidence and mitigates the impact of uncertain predictions. Finally, standardized probabilistic voting first rescales the probabilities per extension and then sums them. This prevents extensions with extreme score ranges from dominating the aggregated result.

Unlike the reliability score component, which aggregates outputs from multiple models for the same review, the voting component aggregates outputs from the same model across all reviews of a given extension to classify the extension.

Note that CoTH does not assign manual weights at any point during this process. The aggregation methods inherently weight reviews either equally (majority) or according to their predicted confidence (unstandardised and standardised).

4.6 Evaluation

Given that our CoTH evaluation implements some models pretrained only in English, the first step in the data pre-processing pipeline involves translating all available reviews into English. In particular, we used Google Translate through an API, although other translation methods

could also have been used. The translation is optimised to exclude non-word comments and truncate comments longer than 5,000 characters to comply with Google Translates API. Note that we therefore have two dataset ore reviews, one entirely in English, and the other one with the original content of the reviews.

To ensure a manageable size and proper balance of our dataset, we adopted a unified approach to resample and reduce the dataset size driven by some empirical analysis of the dataset. In particular, to accomplish this, we implemented a single method that achieves both resampling and size reduction. First, we randomly removed extensions, primarily choosing benign extensions to undersampling the data given the low entropy that they add to the model. Second, we limited the number of reviews per extension to the 100 most recent ones. Note that such a number is derived from the statistical analysis performed in Fig. 1 and ensures that most of the comments of the extensions are used.

The final reduced dataset contains over 15,000 reviews from 1,355 extensions, where 407 are for testing and 948 for training. Although this dataset represents a significant size reduction compared to the original dataset it preserves the essential characteristics of the data. Also, this reduced dataset provides a more manageable and diverse foundation for developing and evaluating our Proof of Concept (PoC) models.

4.6.1 Reliability

The initial phase of our methodology involved using three models from Huggingface Hub, a public repository of ML models. Each model had been pre-trained by its creator to classify online reviews as real or computer-generated by fine-tuning different LLM. We used the models in a collective decision-making process whereby each model contributed to determining a reliability score for individual users by voting. The three voting systems are a majority vote, the highest unstandardised value, and the highest standardised value. Note that for all the experiments presented in this section, we split the dataset into 70% and 30% for the training and the testing dataset.

4.6.2 Models

We initially deployed CoTH using seven pre-trained LLM, two trained on multiple language texts:

i) Twitter-RoBERTa XLM [54];
ii) BERT base multilingual uncase [12].

And five on just English texts:

i) Twitter-RoBERTa Sentiment [53];
ii) Otis Anti-Spam [35];
iii) BERTPhishing [10];
iv) BERT [12], and;
v) distilBERT [42]).

The selection comprises a diverse mix of models trained for different objectives. This enables us to test which of these known NLP objectives and implementations are best suited to be used by CoTH to extend the model for malware detection.

To evaluate the performance of the CoTH architecture, we trained a model for each base LLM for a maximum of 100 epochs, utilising early stopping with a patience of 5 epochs. Training is halted when performance stagnates, indicating that further training may lead to overfitting or negligible improvements. We then calculate precision, recall, F_1 score and Matthew Correlation Coefficient (MCC) for each model in each prediction mode: majority vote, highest unstandardised score and highest standardised scores.

We employed a split Learning Rate (LR) to improve the selected models further. This allowed us to use a small LR for the base LLM, which only requires minor adjustments, and a more aggressive LR for the fully connected layers, which might need more extensive adjustments since it does not already "know" the data. We chose for the base LLM maintained a LR of 2e-5 as previously suggested [48] to overcome the catastrophic forgetting problem and tried multiple values for the CoTH specific fully connected layers to find the best one. After several trials, we chose a LR of 1e-2 for the fully connected layers.

Based on the language, we split the results into multilingual (see Table 4) and English (see Table 5), depending on the data used to train the models.

Focusing on the multilingual setting, the Highest Unstandardised (HU) strategy consistently yields the best recall as well as competitive F_1 and MCC values, particularly for Twitter-RoBERTa XLM, suggesting that it mitigates the impact of differences in base model quality. While the Majority and Highest Standardised (HS) strategies occasionally produce slightly higher precision, this often comes at the cost of reduced recall, especially for BERT. The fact that HU achieves the highest MCC reinforces its advantage, as this metric accounts for class balance and indicates overall robustness rather than improvements in a single dimension. Note that recall remains the most relevant metric for our task, as commonly preferred in malware analysis and health-related research [40].

For completeness, we include the confusion matrices of these two models in Figs. 4 and 5, where we can corroborate that the main effect of the voting system is to shift the

Table 4 Scores by model and configuration using split LR trained on multilingual reviews. HU denotes Highest Unstandardised and HS denotes Highest Standardised

Metric	Base Pre-trained Model	Voting System		
		Highest Unstandardised	Highest Standardised	Majority
Precision	Twitter-RoBERTa XLM	0.480	**0.485**	**0.485**
	bert-base-multi-lingual-uncased	0.590	0.476	**0.614**
Recall	Twitter-RoBERTa XLM	**0.724**	0.710	0.710
	bert-base-multi-lingual-uncased	**0.565**	0.449	0.507
F_1	Twitter-RoBERTa XLM	**0.578**	0.576	0.576
	bert-base-multi-lingual-uncased	**0.577**	0.462	0.555
MCC	Twitter-RoBERTa XLM	**0.485**	0.483	0.483
	bert-base-multi-lingual-uncased	**0.494**	0.357	0.478

balance between precision and recall rather than dramatically change overall accuracy.

Regarding the English version, in Table 5 we can see that no single voting strategy consistently maximises all metrics for all models. For instance, HU usually achieves the highest recall for several models (e.g., Twitter-RoBERTa XLM, bert-base-multilingual-uncased, OTIS-Official-Span-Model, and both BERT-base-uncased and DistilBERT-base-uncased). In contrast, HS and Majority often improve precision, F_1, or MCC scores in other cases.

Notably, multilingual models like Twitter-RoBERTa XLM and BERT-base-multilingual-uncased generally achieve higher recall and MCC than the monolingual or task-specific models, indicating that leveraging multilingual pre-training can improve overall robustness. These results align with the previous experiment using original reviews and indicate that the multilingual models usually outperform the monolingual ones and that there is no need to translate reviews to achieve high recall scores.

4.6.3 Validity

We performed an ablation study to compare the impact of the individual components on the final performance of the CoTH framework. However, due to the architecture, some components are impossible to remove on their own or at all, i.e., the reliability scores and fully connected layers cannot be removed independently of each other, and the voting component cannot be removed at all as it changes the structure of the final output. Therefore, our ablation study includes three modes (see Table 6). “FC-only”, wherein the base LLM is omitted and CoTH is simply a neural network operating on the calculated reliability scores. “LLM-only”, where the fully connected layers and, therefore, the reliability scores are omitted—this is equivalent to simply fine-tuning the base LLM. Lastly, the default mode, CoTH using the multilanguage pretrained models trained with the original reviews—without translation.

When comparing the different approaches, we observe that using CoTH consistently improves recall and MCC compared to using the LLM alone, while slightly reducing precision. Interestingly, CoTH also reduces the performance gap between the underlying models: both achieve similar F_1 and MCC values when combined with CoTH, whereas their performance varies more when used in isolation. In contrast, the FC-only approach performs substantially worse across all metrics, highlighting the importance of leveraging pretrained LLM. The takeaway of this ablation study is that applying CoTH makes the choice of base model less critical while yielding more stable and robust performance.

5 Related work

This section reviews the current state of the art in malware analysis using users’ comments in public repositories. In particular, we start analysing the most important techniques to detect fake reviews and how metadata and users’ comments are used to detect malware.

5.1 Fake reviews

Fake or malicious reviews are not new [32]. However, ongoing research endeavours continue to try and tackle the challenge of effectively addressing these harmful practices. These efforts persist due to the adaptive nature of malicious actors. Whether or not they are human or automated, they will continually try to innovate new techniques to circumvent detection mechanisms. Recent advancements in ML have provided promising avenues for discerning the authenticity of reviews without direct human intervention.

Table 5 Score by model and mode with split LR using English reviews. HU stands for Highest Unstandardised and HS for Highest Standardised

Metric	Base Pre-trained Model	Voting System		
	Prediction	Highest Unstandardised	Highest Standardised	Majority
Precision	Twitter-RoBERTa XLM	0.367	**0.524**	0.360
	bert-base-multilingual-uncased	**0.586**	0.552	0.576
	Twitter-RoBERTa Sentiment	0.448	0.444	**0.454**
	OTIS-Official-Spam-Model	**0.380**	0.358	0.346
	bert-finetuned-phishing	**0.478**	0.466	0.45
	bert-base-uncased	0.452	0.403	**0.487**
	distilbert-base-uncased	0.437	0.404	**0.464**
Recall	Twitter-RoBERTa XLM	**0.763**	0.697	0.75
	bert-base-multilingual-uncased	**0.578**	0.486	0.447
	Twitter-RoBERTa Sentiment	0.26	**0.32**	0.2
	OTIS-Official-Spam-Model	**0.48**	0.38	0.34
	bert-finetuned-phishing	0.22	**0.28**	0.18
	bert-base-uncased	**0.756**	0.609	0.695
	distilbert-base-uncased	**0.768**	0.670	0.707
F_1	Twitter-RoBERTa XLM	0.495	**0.598**	0.487
	bert-base-multilingual-uncased	**0.582**	0.517	0.503
	Twitter-RoBERTa Sentiment	0.329	**0.372**	0.277
	OTIS-Official-Spam-Model	**0.424**	0.368	0.343
	bert-finetuned-phishing	0.301	**0.35**	0.257
	bert-base-uncased	0.566	0.485	**0.572**
	distilbert-base-uncased	0.557	0.504	**0.560**
MCC	Twitter-RoBERTa XLM	0.368	**0.498**	0.355
	bert-base-multilingual-uncased	**0.487**	0.416	0.411
	Twitter-RoBERTa Sentiment	0.223	**0.249**	0.196
	OTIS-Official-Spam-Model	**0.260**	0.203	0.179
	bert-finetuned-phishing	0.220	**0.245**	0.183
	bert-base-uncased	0.445	0.332	**0.452**
	distilbert-base-uncased	**0.435**	0.358	**0.435**

Table 6 Ablation study focusing on individual reviews. "FC-only", where the base LLM is omitted and CoTH is simply a neural network; "LLM-only", where the fully connected layers and, therefore, the reliability scores are omitted—this is equivalent to simply fine-tuning the base LLM, and; CoTH using the Highest Unstandardised voting system. This experiment we use original reviews, i.e., multilingual models

	Pre-trained Model	Precision	Recall	F_1	MCC
FC-only	N/A	0.548	0.249	0.343	0.137
LLM-only	Twitter-RoBERTa XLM	0.625	0.611	**0.618**	0.337
	bert-base-multilingual-uncased	**0.670**	0.492	0.567	0.330
CoTH	Twitter-RoBERTa XLM	0.480	**0.724**	0.578	0.485
	bert-base-multilingual-uncased	0.590	0.565	0.577	**0.494**

As an example, to find malicious reviews, authors used a supervised learning algorithm to classify comments into authentic and fake [8]. The authors scored each review on *understandability*, *level of details*, *writing style*, and *cognition indicators*. Then, they used these scores as features for many machine learning models to find a classifier to distinguish real and fake reviews. They showed that most of them reached an agreeable accuracy based on these features alone.

More recent studies [5, 15, 16, 20, 33] systematically compare the performance of multiple ML models, including Random Forest, Support Vector Machines (SVM), and Multilayer Perceptron (MLP), demonstrating their performance across different datasets. These ML models have a broad range of applicability to accommodate diverse contexts, such as film critiques [16], restaurant, hotel, or other e-commence reviews [5, 15, 20], or browser extensions [33].

Like the techniques used in CoTH, authors analysed users' reviews, such as detecting fake comments using multiple NLP models and their ability to classify user-generated reviews as real or fake [14]. The models evaluated include *Logistic Regression*, *Naive Bayes*, *K-Nearest Neighbour*, *Support Vector Machine* and *Random Forest*. The authors show that all tested models could successfully classify a customer review as real or fake with high accuracy using both bi- and tri-gram versions of the models. This is highly relevant to our setting as users might generate fake reviews to boost the popularity of their extension to garner more downloads propagating their malware [33].

5.2 Fake reviews for malware detection

The use of user reviews specifically for malware detection is far less common [21, 22, 27, 37, 41] and focuses mainly on applying ML or statistical methods to specific types of user reviews to perform their classification. Specifically, these approaches use only reviews associated with a sufficiently low rating and containing specific keywords as specified by the authors [21, 22, 27, 37]. The motivation behind this approach is stated almost unanimously to be that users will detect malicious behaviour and leave poor reviews. They do not take into account malicious actors that try to inflate their perceived popularity artificially [33].

Considering the complete set of reviews is also familiar since [41] uses multiple modules to extract features from the available metadata; of these modules, two use reviews. One module uses the review content and rating to filter out fraudulent reviews and then uses specific keywords to generate its features. The second module forgoes NLP entirely. Instead, it focuses on which users are likelier to leave reviews on the same applications in the same time frame. It aims to identify crowdsourcing of fraudulent reviews to boost apparent popularity and help proliferate malware artificially.

The vast majority of analysis focuses on application permissions [1, 3, 4, 26, 43, 49, 57] and achieves respectable performance even though the difference in permissions for malicious and benign apps is minimal [43]. It is also common to analyse other metadata left by developers such as their name [3], application description and categories [3, 26, 49], and update frequency [3, 49]. Some papers also make use of user-submitted data such as the average score, usually on a scale of 1 to 5 [3, 13, 49].

ADROIT [30] is a method closer to CoTH, where the authors present a method for detecting malware in an Android application store based solely on metadata supplied by that store. Using textual, categorical and numerical features extracted from each application's store page and manifest, the authors can train a model able to identify malicious apps with a 93.67% accuracy. Furthermore, the authors suggest that their method "can easily be extended to any app market, including iOS and Windows markets, due to the independence from the application software".

The most important differences between ADROIT and CoTH are in the dataset and the amount of processed information. In ADROIT, the dataset had almost 40% of malware samples whereas we only have 0.49%. Regarding the information they used for the model, apart from the manifest file of the Android apps, they also processed metadata like the minimum SDK, the minimum size of the screen and the supported CPU types, identification of the developers, the organisation, the locality and the country, the rating score given by the users, or the number of users who voted and a description among other things. In contrast, we just used the public comments written by users in the CWS.

5.3 Malicious browser extensions

Whilst previous work on the detection of malicious browser extensions in Chrome is abundant [2, 14, 23, 25, 39], most methods implement either static analysis (analysing their source code), dynamic analysis (analysing the behaviour in real-time), or a combination of both (hybrid analysis).

Recently, authors analysed extensions' downloads [39] and related it to malicious extensions. By clustering the downloads over time, the authors classified clusters as malicious or benign if the fraction of known malicious extensions in that cluster is above some threshold. The second classifier aims to classify each extension directly based on the downloads time series. The authors show that both classifiers can successfully identify whether a given extension is malicious or benign with great accuracy, proving that analysis of an extension's metadata is a viable approach for finding malicious browser extensions. Instead, in this paper we analyse users' comments to identify malicious extensions and achieve a recall of 0.75, which is almost the same as the authors achieved by clustering downloads patterns (0.80). Rather than a simple and direct comparison of two different methods, we strongly think that the combination of both will definitely increase the malware detection ability.

5.4 Baselines

Although malware detection from user reviews has, to our knowledge, not been done with Deep Learning (DL) previously. Some attempts were done with more traditional ML methods like Logistic Regression and Random Forest [21, 22]. These studies utilised a *DomainCorpus* sourced from the security domain and a random subset of Android APKs from Amazon's Android, though specifics about the dataset were not disclosed. Unfortunately, these datasets are not publicly accessible, preventing us from reproducing the same results or even training CoTH on their data.

To ensure the fairest possible comparison we implemented the baseline models ourselves and trained them using the same datasets as CoTH and used the training set to generate the feature extraction functions. As the original authors leave some room for interpretation this results in three new models we can use to compare CoTH with existing methods. The first model [22], uses two features based on review content, number of reviews and number of users. The second and third models [21], use an additional feature based on the number of reviews and application age where the interpretation of age is the differentiating factor between the two. We used random forests since authors demonstrated

Table 7 Comparison of CoTH and baseline performance. LoRe stands for Logistic Regression, whereas RF for Random Forest

	Precision	Recall	F_1	MCC	FPR
LoRe	0.283	0.294	0.288	−0.060	0.355
RF A	**0.748**	0.421	0.539	0.429	**0.068**
RF B	0.708	0.469	0.564	0.429	0.092
CoTH	0.480	**0.724**	**0.578**	**0.485**	0.159

it was the best model [21]. To further ensure a fair comparison, the Random Forest (RF) models are re-trained 100 times and their performance averaged to mitigate the impact of large performance differences brought on by their random nature.

We can see how CoTH and the baselines, apart from the Logistic Regression model, perform comparably in terms of all-round metrics such as F_1 and MCC (see Table 7). On the other hand, the baselines and CoTH seem to be opposites in terms of precision and recall, this is likely due to the authors of the baseline prioritising overall accuracy instead of narrower metrics. Due to this discrepancy in preferred metric it is difficult to properly compare the models at a metric-by-metric level.

However, we can still see that CoTH performs comparably to previous work although with a shifted priority of metrics, i.e., prioritising those for malware detection [40]. CoTH's strength lies in its high recall and overall balance in performance, as evidenced by its F_1-Score and MCC (see Table 7). Although the precision and False Positive Rate (FPR) could be improved, the model captures a greater and more reliable number of malicious extensions than the baseline models, making it a better overall choice in malware detection.

While CoTH has a higher FPR (0.159) compared to Random Forest A (0.0680) and B (0.0923), this is offset by its significantly higher recall. Note that a higher recall may come with a slight increase in false positives, but this trade-off is acceptable depending on the application, particularly if identifying true positives is a priority like in malware detection.

Note that, although the results might seem the same, there is an important difference: reproducibility [18]. In our proposal, we do not require any private dataset to train the model with. CoTH uses public reviews from the Web Store to get similar F_1-score than previous works but better recall, i.e., reducing the amount of false alarms and therefore the need of manually analysing browser extensions that are indeed benign.

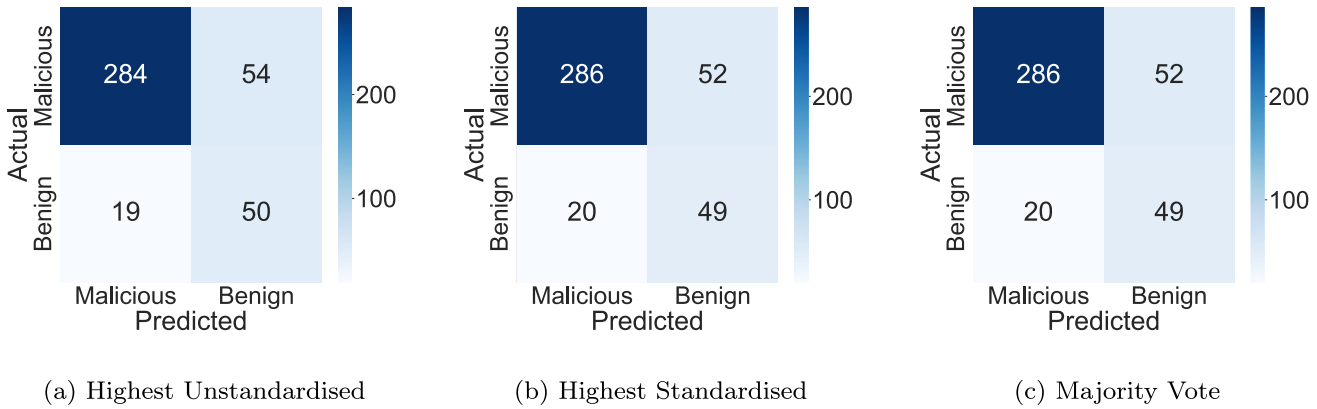
6 Conclusions

In this paper, we have thoroughly analysed comments left on both malicious and benign browser extensions. We have performed an exploratory analysis using TF-IDF, and computing the POS tags of the reviews to later cluster them using HDBSCAN. By doing so, we showed that user reviews left on browser extension store pages do exhibit some weak patterns.

While patterns themselves do not necessarily imply anything about the extensions they are left on, our analysis has further shown that these patterns clearly distinguish comments left on benign and malicious extensions from each other. This further justifies our investigation into applying NLP and DL techniques to classify browser extensions based on their reviews.

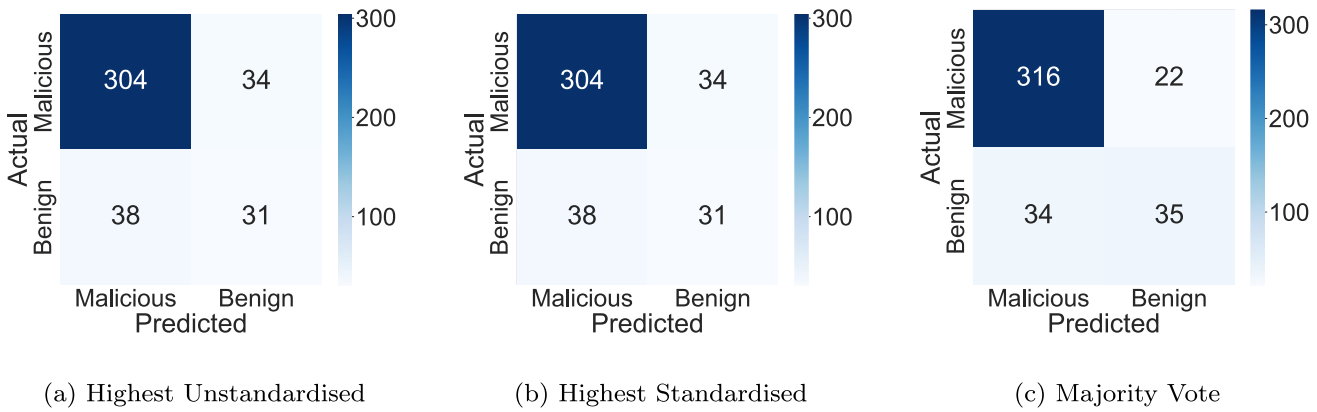
Based on these insights, we proposed CoTH, a novel framework to detect malicious browser extensions by analysing user comments from the Chrome Web Store, a CFRS system. CoTH leverages a combination of pre-trained LLM, traditional neural networks and voting systems to achieve this task. Our experiments demonstrate that the Highest Unstandardised voting strategy consistently maximises recall while maintaining competitive F_1 and MCC scores, particularly for multilingual models such as Twitter-RoBERTa XLM and BERT-base-multilingual-uncased. These models outperform monolingual and task-specific models, showing that multilingual pre-training significantly improves robustness without requiring translation of reviews.

CoTH effectively identifies malicious extensions using only user comments. While our study focuses on reviews from the CWS within a limited timeframe, our results confirm that user-generated content can serve as a reliable signal for malware detection, highlighting the potential of combining advanced language models with strategic voting mechanisms for security tasks in online marketplaces like the Chrome Web Store.



(a) Highest Unstandardised (b) Highest Standardised (c) Majority Vote

Fig. 4 Confusion matrices for Twitter-RoBERTa XLM model with split LR



(a) Highest Unstandardised (b) Highest Standardised (c) Majority Vote

Fig. 5 Confusion matrices for BERT-base-multilingual model with split LR

Model performance

See Figs. 4, 5.

Author Contributions All authors contributed equally to the writing and revision of the paper

Funding Open access funding provided by Halmstad University.

Data Availability No datasets were generated or analysed during the current study.

Declarations

Conflict of interest The authors declare that they have no conflict of interest.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

References

1. Abdulla, S., Altaher, A.: Intelligent approach for android malware detection. KSII Transactions on Internet and Information Systems (TIIS) **9**(8), 2964–2983 (2015)
2. Aggarwal, A., Viswanath, B., Zhang, L., Kumar, S., Shah, A., Kumaraguru, P.: I spy with my little eye: Analysis and detection of spying browser extensions. In: Euro S &P (2018)
3. Akhuseyinoglu, N.B., Akhuseyinoglu, K.: Antiware: An automated android malware detection tool based on machine learning approach and official market metadata. In: UEMCON, pp. 1–7. IEEE (2016)
4. Almin, S.B., Chatterjee, M.: A novel approach to detect android malware. Procedia Computer Science **45**, 407–417 (2015)
5. Alsubari, S.N., Deshmukh, S.N., Alqarni, A.A., Alsharif, N., Aldhyani, T., Alsaade, F.W., Khalaf, O.I.: Data analytics for the identification of fake reviews using supervised learning. Computers, Materials & Continua **70**(2), 3189–3204 (2022)
6. Anderson, B., Quist, D., Neil, J., Storlie, C., Lane, T.: Graph-based malware detection using dynamic analysis. J. Comput. Virol. **7**, 247–258 (2011)
7. Ars Technica: Hola VPN used to perform DDoS attacks, violate user privacy. https://arstechnica.com/information-technology/20

15/06/hola-vpn-used-to-perform-ddos-attacks-violate-user-privacy (2020)
8. Banerjee, S., Chua, A.Y., Kim, J.J.: Using supervised learning to classify authentic and fake online reviews. In: IMCOM, pp. 1–7 (2015)
9. Beddiar, D.R., Jahan, M.S., Oussalah, M.: Data expansion using back translation and paraphrasing for hate speech detection. Online Social Networks and Media **24**, 100153 (2021)
10. BERT-finetuned-phishing: ealvaradob/bert-finetuned-phishing Hugging Face. https://huggingface.co/ealvaradob/bert-finetuned-phishing (2024)
11. Chawla, N.V., Bowyer, K.W., Hall, L.O., Kegelmeyer, W.P.: Smote: synthetic minority over-sampling technique. Journal of artificial intelligence research **16**, 321–357 (2002)
12. Devlin, J., Chang, M.W., Lee, K., Toutanova, K.: Bert: Pre-training of deep bidirectional transformers for language understanding pp. 4171–4186 (2019)
13. Duque, J., Mendes, G., Nunes, L., de Almeida, A., Serrão, C.: Automated android malware detection using user feedback. Sensors **22**(17), 6561 (2022)
14. Elmogy, A.M., Tariq, U., Ammar, M., Ibrahim, A.: Fake reviews detection using supervised machine learning. International Journal of Advanced Computer Science and Applications **12**(1) (2021)
15. Elmogy, A.M., Tariq, U., Mohammed, A., Ibrahim, A.: Fake reviews detection using supervised machine learning. International Journal of Advanced Computer Science and Applications (2021)
16. Elmurngi, E., Gherbi, A.: Detecting fake reviews through sentiment analysis using machine learning techniques. In: IARIA/Data Analytics (2017)
17. Eriksson, B., Picazo-Sanchez, P., Sabelfeld, A.: Hardening the security analysis of browser extensions. In: Proceedings of the 37th ACM/SIGAPP Symposium on Applied Computing, SAC '22, p. 1694–1703. Association for Computing Machinery, New York, NY, USA (2022)
18. Flemark, A., Paulander, A., Picazo-Sanchez, P.: From comments to threats: Leveraging nlp to detect malicious browser extensions (2025). https://doi.org/10.5281/zenodo.17105329
19. How are items ranked in the store? https://developer.chrome.com/docs/webstore/faq/#faq-gen-24 (2024)
20. Gutierrez-Espinoza, L., Abri, F., Namin, A.S., Jones, K.S., Sears, D.R.W.: Fake reviews detection through ensemble learning (2020)
21. Hadad, T., Puzis, R., Sidik, B., Ofek, N., Rokach, L.: Application marketplace malware detection by user feedback analysis. In: ICISSP, pp. 1–19. Springer (2018)
22. Hadad, T., Sidik, B., Ofek, N., Puzis, R., Rokach, L.: User feedback analysis for mobile malware detection. In: ICISSP, vol. 2, pp. 83–94. SciTePress (2017)
23. Jagpal, N., Dingle, E., Gravel, J., Mavrommatis, P., Provos, N., Rajab, M., Thomas, K.: Trends and lessons from three years fighting malicious extensions. In: USENIX (2015)
24. Japkowicz, N., Stephen, S.: The class imbalance problem: A systematic study1. Intelligent data analysis. **6**(5) (2002-11-15)
25. Kapravelos, A., Grier, C., Chachra, N., Kruegel, C., Vigna, G., Paxson, V.: Hulk: Eliciting malicious behavior in browser extensions. In: USENIX (2014)
26. Lemos, R., Heinrich, T., Maziero, C.A., Will, N.C.: Is it safe? identifying malicious apps through the use of metadata and interprocess communication. In: SysCon, pp. 1–8. IEEE (2022)
27. Li, S., Kumar, S., Dumitras, T., Subrahmanian, V.S.: Breaking Bad: Forecasting Adversarial Android Bad Behavior, pp. 405–431. Springer International Publishing, Cham (2018)
28. Livshits, C.C.B., Zorn, B., Seifert, C.: Zozzle: Low-overhead mostly static javascript malware detection. In: USENIX (2010)
29. Maimon, O., Rokach, L.: Ensemble Methods for Classifiers, pp. 957–980. Springer US, Boston, MA (2005)
30. Martín, A., Calleja, A., Menéndez, H.D., Tapiador, J., Camacho, D.: ADROIT: Android malware detection using meta-information. In: SSCI, pp. 1–8. IEEE (2016)
31. Moreno, J.M., Vallina-Rodriguez, N., Tapiador, J.: Did i vet you before? assessing the chrome web store vetting process through browser extension similarity. arXiv:2406.00374 (2024)
32. Mukherjee, A., Liu, B., Glance, N.: Spotting fake reviewer groups in consumer reviews. In: The Web Conference, p. 191–200 (2012)
33. Olsson, E., Eriksson, B., Andersson, L., Picazo-Sanchez, P., Sabelfeld, A.: Fakex: A framework for detecting fake reviews of browser extensions. In: Asia CCS, p. 769–784 (2024)
34. Or-Meir, O., Nissim, N., Elovici, Y., Rokach, L.: Dynamic malware analysis in the modern era-a state of the art survey. ACM Computing Surveys (CSUR) **52**(5), 1–48 (2019)
35. OTIS-Official-Spam-Model: Titeiiko/OTIS-Official-Spam-Model Hugging Face. https://huggingface.co/Titeiiko/OTIS-Official-Spam-Model (2024)
36. Pan, Y., Ge, X., Fang, C., Fan, Y.: A systematic literature review of android malware detection using static analysis. IEEE Access **8**, 116363–116379 (2020)
37. Pantelaios, N., Nikiforakis, N., Kapravelos, A.: You've changed: Detecting malicious browser extensions through their update deltas. In: CCS, p. 477–491 (2020)
38. de Paula, A.F.M., da Silva, R.F., Schlicht, I.B.: Sexism prediction in spanish and english tweets using monolingual and multilingual bert and ensemble models. arXiv:2111.04551 (2021)
39. Picazo-Sanchez, P., Eriksson, B., Sabelfeld, A.: No signal left to chance: Driving browser extension analysis by download patterns. In: ACSAC, pp. 896–910 (2022)
40. Powers, D.: Evaluation: From precision, recall and f-measure to roc, informedness, markedness & correlation. J. Mach. Learn. Technol. **2**(1), 37–63 (2011)
41. Rahman, M., Rahman, M., Carbunar, B., Chau, D.H.: Search rank fraud and malware detection in google play. IEEE Trans. Knowl. Data Eng. **29**(6), 1329–1342 (2017)
42. Sanh, V., Debut, L., Chaumond, J., Wolf, T.: Distilbert, a distilled version of bert: smaller, faster, cheaper and lighter. arXiv:1910.01108 (2019)
43. Sanz, B., Santos, I., Laorden, C., Ugarte-Pedrero, X., Bringas, P.G., Álvarez, G.: Puma: Permission usage to detect malware in android. In: CISIS, pp. 289–298. Springer (2013)
44. Schmidt, A.D., Bye, R., Schmidt, H.G., Clausen, J., Kiraz, O., Yuksel, K.A., Camtepe, S.A., Albayrak, S.: Static analysis of executables for collaborative malware detection on android. In: ICC, pp. 1–5. IEEE (2009)
45. Shijo, P., Salim, A.: Integrated static and dynamic analysis for malware detection. Procedia Computer Science **46**, 804–811 (2015)
46. Spring, T.: Copyfish Browser Extension Hijacked to Spew Spam. Threatpost (2017). https://threatpost.com/copyfish-browser-extension-hijacked-to-spew-spam/127125. [Accessed: 06-12-2023]
47. Statcounter Global Stats: Browser Market Share Worldwide Statcounter Global Stats. https://gs.statcounter.com/browser-market-share (2023)
48. Sun, C., Qiu, X., Xu, Y., Huang, X.: How to fine-tune bert for text classification? In: CCL, pp. 194–206. Springer (2019)
49. Teufl, P., Ferk, M., Fitzek, A., Hein, D., Kraxberger, S., Orthacker, C.: Malware detection by applying knowledge discovery processes to application metadata on the android market (google play). Security and communication networks **9**(5), 389–419 (2016)
50. Thangavelooa, R., Jinga, W.W., Lenga, C.K., Abdullaha, J.: Datdroid: Dynamic analysis technique in android malware detection.

International Journal on Advanced Science, Engineering and Information Technology **10**(2), 536–541 (2020)
51. Titterington, A.: Dozens of malicious extensions for Google Chrome. https://www.kaspersky.com/blog/dangerous-chrome-extensions-87-million/48562/ (2023)
52. Tomek, I.: Two modifications of cnn. IEEE Trans. Syst. Man Cybern. **SMC–6**(11), 769–772 (1976)
53. twitter-roberta-base-sentiment-latest: cardiffnlp/twitter-roberta-base-sentiment-latest Hugging Face. https://huggingface.co/cardiffnlp/twitter-roberta-base-sentiment-latest (2024)
54. twitter-xlm-roberta-base-sentiment: cardiffnlp/twitter-xlm-roberta-base-sentiment Hugging Face (2024)
55. Varshney, G., Bagade, S., Sinha, S.: Malicious browser extensions: A growing threat: A case study on google chrome: Ongoing work in progress. In: ICOIN, pp. 188–193. IEEE (2018)
56. Wang, J., Xue, Y., Liu, Y., Tan, T.H.: JSDC: A Hybrid Approach for JavaScript Malware Detection and Classification. In: AsiaCCS, p. 109–120 (2015)
57. Wang, K., Song, T., Liang, A.: Mmda: Metadata based malware detection on android. In: CIS, pp. 598–602. IEEE (2016)
58. Wang, Y., Cai, W., Lyu, P., Shao, W., et al.: A combined static and dynamic analysis approach to detect malicious browser extensions. Security and Communication Networks **2018** (2018)
59. Wilson, D.L.: Asymptotic properties of nearest neighbor rules using edited data. IEEE Trans. Syst. Man Cybern. **SMC–2**(3), 408–421 (1972)
60. Xing, X., Meng, W., Lee, B., Weinsberg, U., Sheth, A., Perdisci, R., Lee, W.: Understanding malvertising through ad-injecting browser extensions. In: The Web Conference, p. 1286–1295 (2015)
61. Xue, L., Zhu, T.: Hybrid resampling and weighted majority voting for multi-class anomaly detection on imbalanced malware and network traffic data. Eng. Appl. Artif. Intell. **128**, 107568 (2024)
62. Zimperium: The Case of Cloud9 Chrome Botnet - Zimperium. https://www.zimperium.com/blog/the-case-of-cloud9-chrome-botnet (2022)

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.